

Analysis: Candy Store

Summary

At first this problem seems intimidating. There are a huge number of ways customers can order, and even once you've decided which boxes to order, verifying that all possible scenarios are covered is non-trivial.

But as it turns out, a greedy algorithm for choosing the boxes, and a greedy approach to handing them out, ends up working! And as with most greedy problems, the difficulty lies only in convincing yourself that it works. It's actually the easiest problem in the set to implement. The only thing you must overcome is self-doubt!

Proof

Suppose that part of our order is a set of boxes with total weight N grams, with no box larger than $X = \text{floor}(N/k) + 1$. Furthermore, suppose that we know these boxes have the property that if the k customers' orders don't total more than N grams, these boxes can be used greedily to satisfy their orders. "Greedy" means that when a customer comes to the store, you simply find the largest box that is less than or equal to his order, and give it to him, repeating until you've filled his order exactly. This is our inductive hypothesis: a guarantee that this strategy works.

Now, suppose we order another box with weight X (ie, $\text{floor}(N/k) + 1$) grams, making a new set of boxes with total weight $N + X$ grams (and note that no box is larger than $\text{floor}((N + X)/k) + 1$, trivially). Suppose the k customers order no more than $N + X$ grams. What happens if we apply the greedy strategy? Well, as soon as we see an order of $\geq X$ cents, we will use this new X -gram box on it immediately. If we now pretend that the customer instead ordered X fewer cents, then we know the greedy strategy with the rest of the boxes works. The choices made by the strategy are unchanged aside from the use of the new box. If it turns out that there is NO order of $\geq X$ cents, then the total is at most $k \cdot (X - 1) \leq k \cdot (N/k) = N$ grams. The new box can't ever be used, so the greedy strategy does the same thing for these orders that it did before, and thus it works.

So using induction, we now have what I'll call "The Algorithm", which constructs sets of boxes that work greedily. We start with 0 boxes, and keep adding new boxes of size $\text{floor}(\text{"total sum so far"}/k) + 1$. eg, first we'll add k boxes of size 1 (which we obviously need), then a box of size 2, and so on.

Ok, now how do we prove that this is the best we can do? Suppose we've ordered some set S of boxes that works. Sort the boxes from smallest to largest, and consider the first box that is larger than would be chosen by "The Algorithm" based on all the boxes smaller than it. In other words, if this box is of size Y , and N is the total of all the boxes smaller than it, $Y > X = \text{floor}(N/k) + 1$. (Note that $X \leq C$ since $N < k \cdot C$.) What happens if the customers all order X cents? None of the boxes of size Y or greater can be used, and the total is $k \cdot X > N$, so all the boxes smaller than Y don't add up to enough to handle them. This can't possibly work, regardless of strategy.

Thus, every box in S , considered in sequence, is no larger than would be added by "The Algorithm" based on the previous boxes. But if any box chosen is STRICTLY smaller, then "The Algorithm"'s later selections will all be reduced accordingly (it is monotonic in that sense). Since S must have sum at least $k \cdot C$, if we ignore S and instead order an equal number of boxes using

"The Algorithm", their sum will be at least $k \cdot C$ as well. This proves that "The Algorithm" is the best we can do.

Implementation

"The Algorithm" is really, really simple to code. In fact, this is a complete implementation of a Candy Store solution:

```
long long T, k, C, prob = 1;
for (cin >> T; T--;) {
    cin >> k >> C;
    long long sum = 0, num_boxes = 0;
    while (sum < k*C) {
        num_boxes++;
        sum += (sum / k) + 1;
    }
    cout << "Case #" << prob++ << ": " << num_boxes << endl;
}
```

Other Cool Stuff

The observant among you may have noticed that this solution is very general, and remains unchanged even if we vary the problem a little. For instance:

- The bound C is effectively useless. We can throw it out and let customers order any amount, as long as the TOTAL order is no more than $k \cdot C$.
- We can do no better even if we're told all the customers' orders in advance (at the beginning of the day). The "online" solution is just as good as an "offline" one.
- We can do no better even if we constrain the customers to all order the same amount as each other. This was actually the original proposed form of the problem.

Analysis: City Tour

This proved to be the easiest problem in the finals for the experienced competitors.

We get some insight into the problem by noticing the graph resembles a tree. These graphs are actually called partial 2-trees. We can actually build a tree if we follow the way the graph is constructed. We build another graph that associates a node to each new added cycle of length this cycle shares an edge to an old cycle so we add an edge between the two respective nodes in the second graph. This way we have built the tree decomposition of these graphs. As on trees many problems that are hard to solve on general graphs have polynomial algorithms on this type of graphs.

Let's solve the related problem of finding the longest path in a tree. We can use dynamic programming and depth first search. We mark a node as a root. Every path in the tree has exactly one node that is closest to the root and this node splits the path in two downward paths. Now for each node in the tree we compute the longest downwards path that starts in it. To find the largest path in the tree we look at each node and at the two longest paths that start in it's children. This solves the problem in linear time.

The solution for our original problem is pretty similar. For each edge (x,y), we compute the cycle that contains it and all the other nodes in the cycle have larger indexes. Let's call this a downward cycle since it goes the opposite direction of where the initial three nodes are. To find that number we have to look at all higher indexed nodes that were connected to this edge and try to use them as intermediary points in the cycle. So for a given intermediary point z we can build a cycle by looking at the longest downward cycle that contains the edge (x,z) and the longest downward cycle that contains the edge (z,y), use all the edges, add edge (x,y) and remove the edges (x,z) and (z,y).

We also compute the largest downward cycle which contains these two nodes but doesn't contain this edge, this is a union of the cycle that goes through these nodes and the second largest path from which we remove the edge (x,y).

And here is some code that does implements this solution:

```
int best_so_far = 0;

int best(int x, int y, int N, int[][] a) {
    int max_len = 2;
    int second_max_len = -1;
    for (int i = Math.max(x, y) + 1; i < N; i++) {
        if (a[x][i] * a[y][i] > 0) {
            int len = best(x, i, N, a) + best(y, i, N, a) - 1;
            if (len > max_len) {
                second_max_len = max_len;
                max_len = len;
            } else if (len > second_max_len) {
                second_max_len = len;
            }
        }
    }
    best_so_far = Math.max(max_len, best_so_far);
    best_so_far = Math.max(max_len + second_max_len - 2,
                           best_so_far);
}
```

```
    return max_len;  
}
```

Another cool solution is based on the idea of contracting each node of degree two. We replace it with an edge which has the weight equal to the sum of the weights of the two incoming edges. It's a pretty neat idea and we'll let you figure out the details on your own.

Analysis: Letter Stamper

The problem idea originated from the report of one of Robert Tarjan's talks, where a quadratic solution for four letters was claimed. We found the easier three-letter version was already very interesting, and presented it to you as the first problem for this Code Jam final.

We note that the general case where the alphabet size is not restricted to a small constant such as 3 can be solved by dynamic programming in $O(n^3)$. It had appeared in several programming contests before.

The current state is characterized by a pair (s, κ) , where s is the suffix of the input string you still need to print (so the next letter you need to print is the head of s), and κ is the stack. For each situation, we need to decide if the next step is a push, a pop, or a print.

We state some rules that are satisfied by an optimal sequence. These will make the solution quite clear. There can be other optimal sequences, but you can always reduce them to one that satisfies these conditions.

Rule 1. If the first letter of s is the same as the top of κ , then the next step is a print.

Rule 2. You never need to push the letter that's already on top of the stack. Therefore no letters appears on the stack twice in a row.

Rule 3. Immediately after you push a letter x , the next step is to print x .

The rules above are trivial and we omit the reasoning. But it might worth spending a little time convincing yourself formally.

Rule 4. There should never be three consecutive letters in the stack of the form xyx .

Let us justify this rule. Suppose an optimal solution has XYX on the stack. Let's modify it. At some point the top of the stack is XY and we were going to push another X . Instead, we will pop the Y . Then we continue as before, until we were going to pop the second X that now doesn't exist. Instead, we will push Y , arriving at the same state as before. This way we have shortened the stack while achieving a solution of the same length. By induction you can keep simplifying the solution until all 4 rules are satisfied.

Rule 4, together with Rule 2, implies that the stack always contains a cycle of 3 letters, e.g. $ABCABCABC \dots$. The state of the stack then is completely defined by the first two letters and the stack height. There are only 6 patterns, and the height of the stack is never bigger than n . Hence the number of possible states (s, κ) is $O(n^2)$. This gives a quadratic dynamic programming solution.

Analysis: Ninjutsu

Getting something discrete

A fact that jumps out immediately is that this problem is not discrete. The rope length that gives an optimal solution might be a real number, and there does not seem to be a way to ensure that this number is rational or discrete in any other way.

Given any real-valued rope length, we can simulate and count the bends, but this number is not a convex function of the rope length, so ternary search is out.

What we really wish we could do is dynamic programming, where the state is a triple -- the point we are currently swinging around, the direction the end of the rope is pointing in, and the length of the rope. Two of these values are real numbers. However, let's see how many different real values we actually need to care about.

As we are swinging the end of the rope around some point **P**, nothing interesting happens until the moving segment of the rope touches another point, **Q**. At that... point, we need to make a decision -- we could either bend the rope and switch to swinging around point **Q**, or cut the rope at exactly point **Q** and continue swinging around **P**. There are at most N^2 pairs of points, so we only need to consider at most N^2 different directions. That takes care of the second DP parameter.

What about the rope length? Using the same reasoning as above, we can show that there is only a finite number of rope lengths that are interesting. Firstly, note that bending the rope does not change its length. Whenever we swing around point **P** and hit point **Q**, there is a binary decision -- is the rotating rope segment longer than the distance from **P** to **Q**, or shorter?

In other words, the interval of real numbers between 0 and **R** can be split into a finite number of sub-intervals, such that the number-of-bends function has a constant value on each sub-interval. This observation suggests the following, naive dynamic programming (DP) solution. The DP state is a pair of points (**P**, **Q**) and a real-valued rope length, **r**. The answer is the maximum number of bends we can achieve by continuing to spin around point **P**, if we are currently pointing in the direction of point **Q** and have a swinging segment of the rope of length **r**. To be a bit more precise, we will actually need to have twice as many states -- one for pointing in the direction of **Q**, and one for pointing 180 degrees away from **Q**. We need the second kind of state for the situation when we decide to bend the rope and continue swinging around **Q**. At that point, we will start pointing in the direction 180 degrees away from **Q**. One way to implement this solution is simply to use a point, a 2D vector and a length as the state, instead of two points and a length.

Next, let's get rid of the real-valued length as a DP parameter. Because rope length only changes when we decide to cut it, we can replace the real-valued length parameter with an integer -- the number of bends since the last time we have cut the rope. Let's also replace the first two parameters with the pair of points that caused the cut. We are now interested in all situations of the following sort -- we were swinging around point **P**, with the end of the rope pointing in the direction (**dx**, **dy**), at which point we cut the rope, and since then, we have made **K** bends. This information is enough to identify a state uniquely, and it implies a particular remaining rope length. Our DP state now becomes a triple of **P**, (**dx**, **dy**) and **K**.

Floating point issues

We now have an almost completely discrete problem. The only place where floating point numbers are necessary is in the testing of whether the current rope length is long enough to hit some given point, or does the rope's end pass underneath that point? Given the guarantee that the optimal solution works for a long range of rope lengths (0.999999), we can avoid floating point rounding trouble by being conservative: only assume that the rope can hit a point if its length is longer by at least, say, 0.5 than the distance to the point. If it is shorter than that, then certainly such a solution doesn't work for a long enough range of rope lengths, so it's not the optimal solution by the guarantee.

Dealing with loops

The naive solution is too slow. Consider, for example, the case where we have a rope of length 10^9 and two points: (0, 0) and (0, 1). The optimal solution uses the full length of the rope to create $10^9 - 1$ bends. Of course, we are just going around in loops, so we need a way to detect and handle such loops to have a chance at a polynomial-time solution.

First, let's reorganize the naive DP solution a bit to make loop detection easier. We will have a simpler, three-parameter state and use a memoized recursive function. Each call of the function will correspond to a situation of the following kind -- we are spinning around a point **P**, with the end of the rope pointing in the direction (**dx**, **dy**), and we have just cut the rope because it has touched another point, **Q**. The total number of such states is $O(N^3)$, but many of them are not possible and will never be visited.

Inside the function, we will simulate the wrapping of the rope and make recursive calls in situations when we decide to cut the rope further. If we simulate the wrapping process naively, we may need to make a huge number of recursive calls. Instead, imagine that we find ourselves in a situation where we have just bent the rope around some point **A**, and we are about to bend it again around some point **B**. If this is not the first time we have seen the pair (**A**, **B**), then we are looping around the same set of points. The second time we hit this pair, we will have a shorter remaining rope length, **r**, and we can figure out how much rope one revolution consumes by subtracting the new value of **r** from the value we had when we first encountered the pair.

Once we detect a loop, we can choose the number of full revolutions we want to make before we cut the rope and enter inside the convex hull of the points we are looping around. Clearly, it never makes sense to cut off more than one whole loop perimeter. We would be throwing away free rope bends. The optimal number of revolutions is, thus, the total remaining rope length, divided by the perimeter of the loop, floored, minus one.

Putting together a complete solution

We now have a DP solution with $O(N^3)$ states and $O(N^2)$ work per state (linear number of recursive calls; linear amount of work to find the next point for each call). We can speed this up by precomputing next points for each state, but this is not necessary. It turns out that the number of reachable states is small enough for this solution to pass.

Remaining difficulty is in dealing with collinear points. Since all coordinates are integers, this should be done exactly, without using floating point computations.

Analysis: The Paths of Yin Yang

*Plenty and void grow from each other;
hard and easy foster each other;
long and short shadow each other;
elegance and mudanity prosper together;
music and voice complement each other;
back and front stay together;
they last forever.*

This version, translated by our colleague Yifan Shi, is certainly closer to the original meaning of the text.

Impressions

One may discover a series of observations in attacking this problem. Some are obvious, while others need some hard work and inspiration. Our algorithm searches all the possible solutions and counts them. Imagine yourself to be a detective facing such an empty rectangular board. Your job is to reveal all the possible solutions, quickly.

For any solution, as the problem stated, almost all the points have degree (number of neighboring squares with the same color as itself) 2; and there are exactly 4 points having degree 1. Let us call the points with degree 1 *end points*;

Once we have decided all the degrees, we can try to reconstruct the color of each square by brute force search. Suppose in a general case, we have colored a square A , and we also colored all but only one of its neighbor B , then we do not need to try both black and white for B , it is decided by the color of the other squares, and the degree of A .

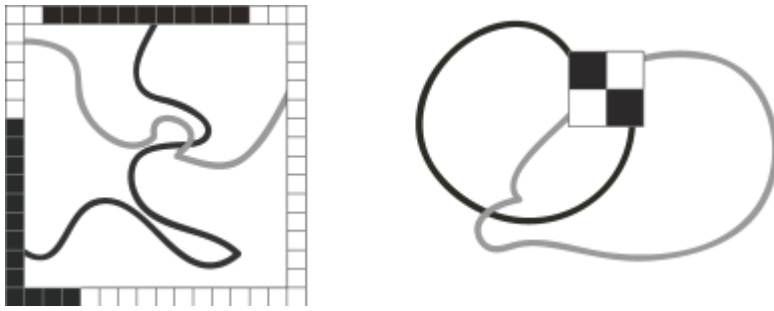
This gives us a much faster way to recover all the solutions than searching blindly. One can see that if all the colors of the first row are decided, and we proceed from top down, then in each step we always have a square (in fact close to M squares) who is in the same situation as A above. So, once we decided the combination (color of the first row, the position of the 4 end points), all we have to do is to check in $O(NM)$ time if a solution arises.

There are 2^M ways to color the first row, and $\Theta((NM)^4)$ ways to pick the end points. Both numbers are too big. One may easily see most of the 2^M colorings of the first row will obviously fail to give a solution, as we will discuss in the next section. After that, we will work on how to reduce the number of positions for the end points.

A bit of Topology

Rather than looking at the first row, let us consider the *outer loop* of the board. That is, the positions on the boundary. It is easy to see that they cannot be of the same color -- the outer loop must contain both black and white squares.

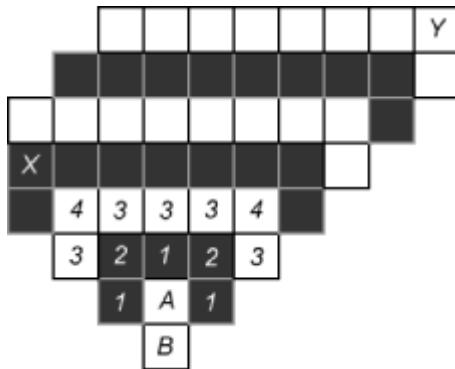
We can say much more about the outer loop. It must be exactly one segment of black squares, and the rest is exactly one segment of white squares! The reason can be seen from the picture on the left below. If there are at least two black segments (and hence at least two white segments), there is no way one can connected both the black pieces and white pieces from the inside of the board.



The second picture shows a forbidden configuration for a 2 by 2 square. If we have a chessboard-like situation, there is no way to connect both black squares and white squares.

Position for end points

Suppose A is a white end point, and B is its sole white neighbor, as in the picture below.



- Because A has no other white neighbors, so the positions labeled with 1 must be black.
- To avoid 2 by 2 chessboard, the positions labeled with 2 must be black as well.
- Because the first row labeled with 1 and 2 has no other black neighbors, so the positions labeled with 3 must be white.
- To avoid 2 by 2 chessboard, the positions labeled with 4 must be white as well.

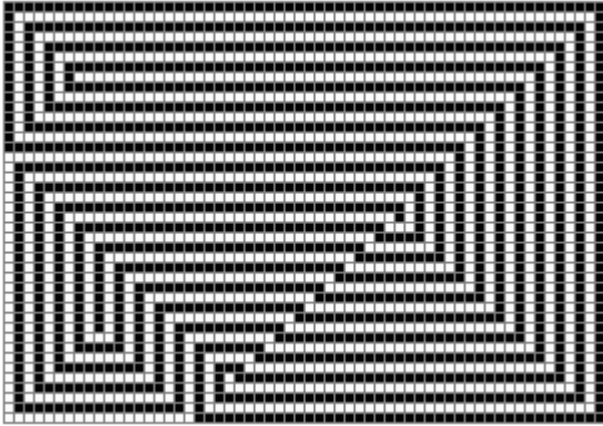
This deduction can go on until we hit the boundary of the board. In our picture X is a point on the boundary, and it is also the place where black and white squares meet on the boundary. Yet we can continue the deduction along the other direction, until we get a point Y on the boundary as well.

This gives a nice connection between the end points and the outer loop. From an end point, we draw two diagonal rays (both with 135 degrees to its neighbor with the same color), both rays hit the boundary either at a corner, or at a point where the black and white squares meet on the outer loop.

Let us summarize our algorithm. We start by fixing the outer loop as a segment of black squares and the rest white. Then from each corner we draw a diagonal line; also from each point on the outer loop that is next to a different color on the loop, we draw one diagonal line going away from the neighbor with the different color. Thus we have 8 diagonals. These diagonals will produce no more than 16 intersections. Those are all the possible positions for the end points.

There are $O((N+M)^2)$ ways to fix the outer loop, and thus $O((N+M)^2)$ ways to fix the outer loop and the end points. For each of these, one can check if a solution arises in $O(MN)$ time. So the running time is $O(MN(N+M)^2)$. The constant is not very small, but tolerable. And one can reduce it using symmetry.

Below is one of a good solution. We may see how the end points and outer loop are connected by the diagonals.



Analysis: Travel Plan

As with City Tour, this problem is closely related to [Hamiltonian cycles](#). The good news is that here, the graph consists only of points on a line, and is therefore much easier to analyze. The bad news is we aren't looking for just any Hamiltonian cycle, or even the shortest one; we are looking for one of a particular length. A question this precise requires a complete search of some kind.

The simplest approach is to try all possible travel plans one at a time. This is much too slow for the large input, but it is fine for the small input.

Given that $N \leq 30$ throughout, you might next think to try a dynamic programming solution that tracks how far you have gone and which nodes you have visited so far. Unfortunately, this is also too slow - large values of F are a big problem!

A 3^N solution

The first key step to solving this problem is to go from $N!$ time to 3^N time.

We first divide the line into intervals $I_1, I_2, \dots, I_{N-1}$. Let t_j denote the number of times that the travel plan crosses interval I_j , and let d_j denote the length of interval I_j . Then the total length of the travel plan is exactly $t_1 * d_1 + t_2 * d_2 + \dots + t_{N-1} * d_{N-1}$. So this means that all we need to do is figure out what each t_j should be.

The big question is what t_j values are possible? Well, let's take a look.

- Each t_j must be positive.
Reason: If $t_j = 0$, then the travel plan never crosses the corresponding interval, and therefore cannot visit every planet.
- For each j , $t_j - t_{j-1}$ must be -2, 0, or 2.
Reason: Let P denote the planet between the intervals I_{j-1} and I_j . In our travel plan, we can stop at P at most once. Every other time, we must travel through, which contributes 1 to both t_{j-1} and t_j . The one time we do hit the planet, we come in along one interval, and then leave along one interval. (These two intervals may or may not be the same.) This part of the travel plan contributes either (a) 1 to both t_{j-1} and t_j , or (b) 2 to one of these values, and 0 to the other. Regardless, $t_j - t_{j-1}$ must be -2, 0, or 2.
- t_0 and t_{N-1} must both be 2.
Reason: Consider the "outside interval" I_{-1} that is past the furthest planets. The travel plan cannot traverse this interval, so $t_{-1} = 0$. The observation here now follows immediately from the previous two.
- Every choice of t_j satisfying the previous conditions can be achieved by a legal travel plan.
Reason: We can construct such a travel plan by scanning from left to right across the intervals. At any given time during the construction, we will have decided what the travel plan does to the left of some planet P . Specifically, for each trip left of P , we will know what the trip does until it goes right of P again. We will not however know what happens between these visits, or even what order they occur in.
Now, suppose we want to extend our partial travel plan past the next interval I_j . If $t_j =$

$t_{j-1} = 2$, then we need to merge two of our current trips via a stop to P , and extend the rest along I_j . If $t_j = t_{j-1} + 2$, then we add a new trip that goes along I_j , stops at P , and then goes back again. In the final case we just extend each trip along I_j , having one stop at P without changing direction.

If you think about this for a while, you should be able to convince yourself that this method does indeed generate a valid travel plan.

Okay! We can now describe a 3^N time solution to the original problem. Given t_{j-1} , there are at most 3 possible choices for t_j . Trying each one in turn, we can iterate over all possible assignments for the t_j 's, and then pick the one that leads to the optimal length travel plan.

A $3^{N/2}$ solution

Even this 3^N time algorithm is too slow however. Fortunately, there is a very handy trick for situations like this.

Let's consider the middle interval $I_{N/2}$ and some fixed value for $t_{N/2}$. Using the approach described above, we can enumerate in $3^{N/2}$ time all choices for $t_1, t_2, \dots, t_{N/2-1}$ that are consistent with $t_{N/2}$. Let **A** be the set of all possible values $t_1 * d_1 + t_2 * d_2 + \dots + t_{N/2} * d_{N/2}$ that can be achieved in this way.

Similarly, we can calculate **B**, the set of all possible values of $t_{N/2+1} * d_{N/2+1} + t_{N/2+2} * d_{N/2+2} + \dots + t_{N-1} * d_{N-1}$ that are consistent with $t_{N/2}$. All this takes $3^{N/2}$ time. What's left is to find two numbers in **A** and **B** whose sum is as close as possible to **F** without exceeding it. One efficient way to do this is to sort **B**, and then for each element in **A**, use binary search to decide which element in **B** you should try matching it with. That gives a $O(3^{N/2} * N)$ solution.

This can be improved to $O(3^{N/2})$ by rearranging the formula a little: $t_1 * (d_1 + \dots + d_{N/2}) + (t_2 - t_1) * (d_2 + \dots + d_{N/2}) + \dots + (t_{N/2} - t_{N/2-1}) * d_{N/2}$. The list of these sums can be generated in sorted order iteratively by repeated merging while adding new terms. The difference $t_{j+1} - t_j$ is always one of -2, 0, 2, so we need to merge 3 sorted lists to add a new term.

When we have the two lists **A** and **B** in sorted order, we can process them in linear time with a single scan using two pointers.